

Introduction to Clustering Algorithms.....	1
What is DBSCAN?	2
Why is DBSCAN Required?	2
Why DBSCAN is Effective	3
Advantages of DBSCAN	3
Meaning of Density in DBSCAN	3
Mathematical Representation	4
Example	5
Why is DBSCAN Different?	5
Differences: K-Means vs DBSCAN.....	6
Real-Life Analogy: People at a Festival	7
Mapping the Analogy to DBSCAN.....	7
Step-by-Step Explanation	7
Clustering Behaviour of DBSCAN	8
Parameters of DBSCAN	8
Epsilon (ϵ or Eps)	9
Impact of ϵ :	9
Impact of Epsilon.....	9
Minimum Points (MinPts)	9
Types of Points in DBSCAN	10
Core Point	10
Border Point.....	10
Noise Point.....	10
Step-by-Step Working of DBSCAN.....	11
Density-Based Concepts	12
Directly Density-Reachable.....	12
Density-Reachable	12
Density-Connected	12
Optimising Epsilon Using the K-Distance Method	13
Steps.....	13
DBSCAN has the following limitations:	14
Comparison of Clustering Algorithms.....	14
Implementation of DBSCAN in Python	15

Required Libraries	15
Generate dataset.....	15
Visualisation	15
Feature Scaling	16
Applying DBSCAN	16
Understanding Output.....	16
Visulisation	17

Introduction to Clustering Algorithms

Clustering is an **unsupervised learning technique** used to group similar data points based on their inherent structure.

Some of the most widely used clustering techniques include:

- K-Means Clustering
- Hierarchical Clustering
- DBSCAN
- Fuzzy C-Means

While these methods are effective in many practical scenarios, they come with several limitations:

- Sensitivity to outliers – even a single extreme value can distort results
- Assumption of cluster shape – expect compact, well-separated clusters
- Poor performance on non-spherical clusters
- Difficulty with overlapping clusters
- Scalability issues (especially hierarchical clustering)

These challenges led to the development of density-based clustering methods, which focus on identifying dense regions separated by sparse areas.

What is DBSCAN?

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a clustering algorithm that:

- Groups closely packed data points into clusters
- Marks isolated points as **noise (outliers)**

Why is DBSCAN Required?

Traditional clustering algorithms such as K-Means work effectively when clusters are approximately:

- Circular or spherical
- Similar in size
- Clearly separated
- Free from extreme outliers

However, real-world data may contain:

- Curved clusters
- Irregularly shaped clusters
- Outliers
- Noise
- Unknown numbers of clusters

DBSCAN was designed to handle many of these situations

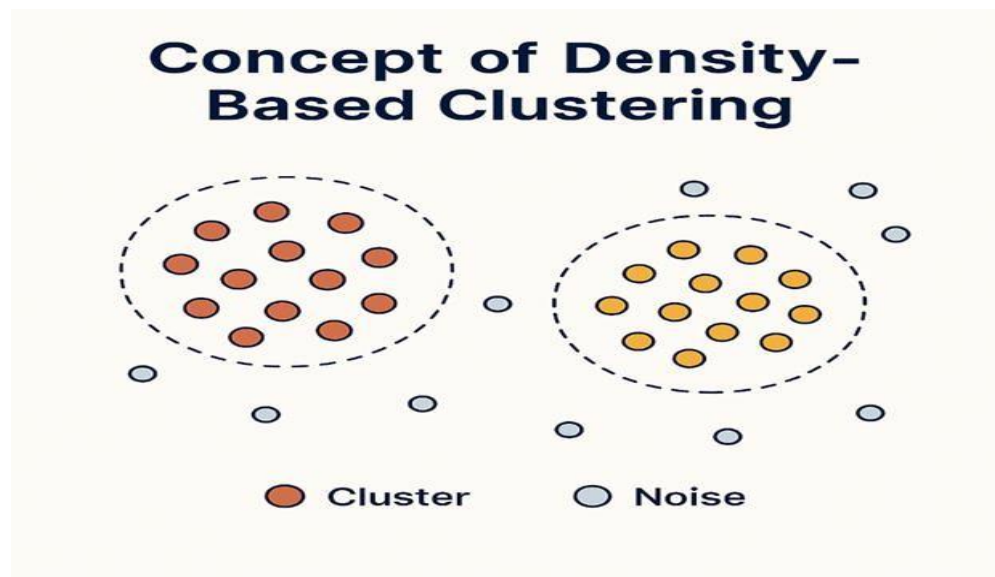


Fig: Dense regions form clusters, while sparse regions represent noise.

Why DBSCAN is Effective

DBSCAN overcomes traditional clustering limitations:

- No need to specify the number of clusters
- Robust to outliers
- Detects clusters of arbitrary shape
- Can separate connected dense regions and identify noise without requiring cluster centroids
- Works well with real-world irregular data

Advantages of DBSCAN

- Robust to noise and outliers
- Handles arbitrary-shaped clusters
- No need to predefine number of clusters
- Effective for large datasets
- DBSCAN is generally most effective in low- to moderate-dimensional datasets. High-dimensional data often requires dimensionality reduction before clustering

Meaning of Density in DBSCAN

In DBSCAN, density refers to the concentration of data points within a specified neighborhood. A region is considered dense when a sufficient number of data points are located close to one another.

DBSCAN measures local density using two parameters:

- **Epsilon (eps or ϵ):** Defines the maximum radius of the neighborhood around a data point.
- **Minimum Points (MinPts or min_samples):** Defines the minimum number of points required within the ϵ -neighborhood to form a dense region.

Simple Meaning: A data point lies in a dense region when it has enough neighboring points within the specified epsilon distance.

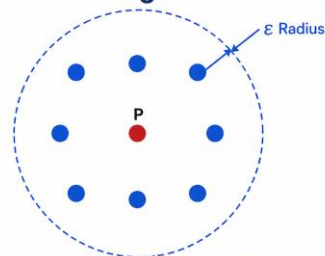
Mathematical Representation

The epsilon neighborhood of a point p is defined as:

$$N_{\epsilon}(p) = \{q \in D \mid d(p, q) \leq \epsilon\}$$

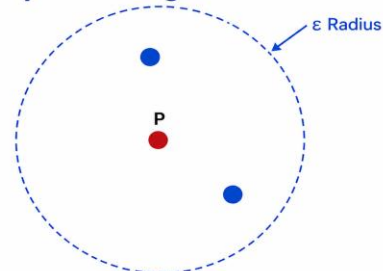
Symbol	Meaning
(D)	Complete dataset
(p)	Selected data point
(q)	Another data point
(d(p,q))	Distance between points (p) and (q)
(\varepsilon)	Maximum neighborhood radius
(N_{\varepsilon}(p))	Points located within ϵ distance of (p)

Dense Neighborhood



● P = Selected Point
 ● Nearby points = 8
 MinPts = 5
 $8 \geq 5$
 ✓ Result: P is a Core Point

Sparse Neighborhood



● P = Selected Point
 ● Nearby points = 3
 MinPts = 5
 $3 < 5$
 ✗ Result: P is not a Core Point

Example

Assume:

$$\epsilon = 2\text{meters}, \text{MinPts} = 5$$

If Point A has six points, including itself, within a distance of two meters:

$$6 \geq 5$$

Therefore, Point A satisfies the density condition and is classified as a **core point**.

If Point B has only three points within the same neighborhood:

$3 < 5$ Point B does not satisfy the core-point condition. However, it may still become a **border point** if it lies within the epsilon neighborhood of another core point.

Important: In Scikit-learn, `min_samples` includes the selected point itself. Therefore, `min_samples=5` means the selected point plus at least four neighboring points are required to satisfy the core-point condition.

Why Is DBSCAN Different?

DBSCAN is different from many traditional clustering algorithms because it defines clusters using **density connectivity** rather than centroids or cluster averages.

K-Means attempts to divide data according to the distance from cluster centroids. As a result, K-Means generally works best when clusters are compact, approximately spherical, and relatively similar in size.

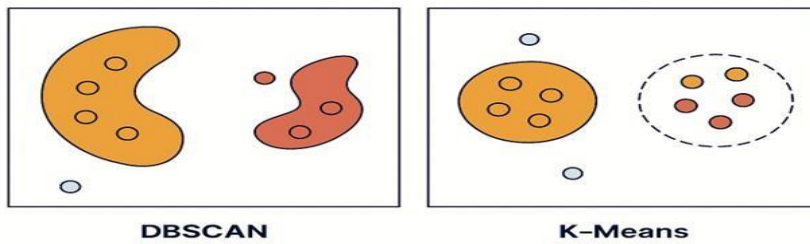
DBSCAN does not calculate centroids. It starts from dense points and continuously expands a cluster by connecting neighboring dense regions. Because of this approach, DBSCAN can discover curved, irregular, and non-convex cluster shapes.

DBSCAN also treats noise as a separate concept. Instead of forcing every observation into a cluster, it can assign isolated observations a noise label.

Differences: K-Means vs DBSCAN

Feature	K-Means	DBSCAN
Clustering Approach	Centroid-based algorithm that groups data points around cluster centers	Density-based algorithm that groups closely packed data points
Number of Clusters	The number of clusters (K) must be specified before training	Automatically discovers clusters based on data density
Cluster Shape	Works best with compact, circular, or spherical clusters	Can identify curved, irregular, and arbitrarily shaped clusters
Outlier Handling	Assigns every data point to a cluster, including possible outliers	Identifies isolated data points as noise or outliers using the label -1

DBSCAN vs K-Means



Real-Life Analogy: People at a Festival

Imagine observing people at a large music festival from above.

Some areas contain many people standing close together:

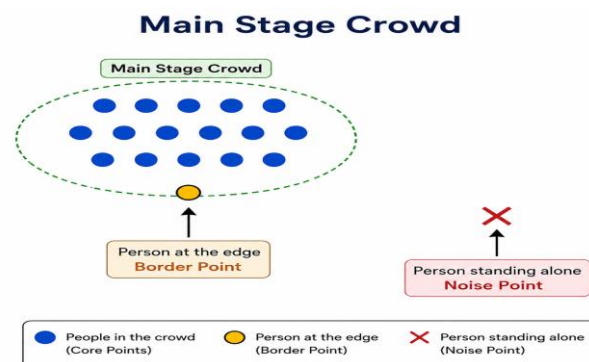
- Near the main stage
- Around food stalls
- Near popular attractions

Other individuals may be walking alone between these crowded areas.

DBSCAN behaves like an observer who identifies crowds based on local density.

Mapping the Analogy to DBSCAN

Festival Concept	DBSCAN Concept
Group of people standing close together	Cluster
Maximum distance used to search nearby people	Epsilon
Minimum crowd size	MinPts
Person surrounded by many people	Core point
Person standing at the edge of a crowd	Border point
Person standing alone	Noise point
Entire connected crowd	Density-based cluster



Step-by-Step Explanation

Assume:

Epsilon = 3 meters

MinPts = 5 people

DBSCAN examines one person.

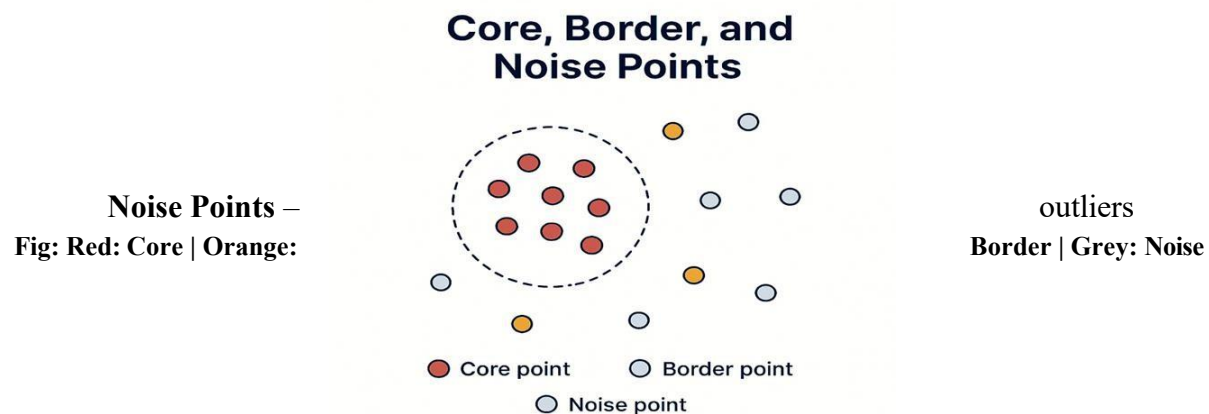
- If at least five people, including the selected person, are located within three meters, the selected person becomes a core point.
- Nearby core points connect with one another and expand the crowd cluster.
- A person near the edge may not have five neighbors but can still belong to the crowd if they are located within the epsilon neighborhood of a core person. Such an individual represents a border point.
- A person far away from all crowds may be classified as a noise point.

Clustering Behaviour of DBSCAN

DBSCAN identifies clusters by examining the **local density** of data points. It groups closely packed observations into clusters and separates dense regions using low-density areas.

Unlike K-Means, DBSCAN does not assign every observation to a cluster. It classifies each data point into one of three categories:

- **Core Point:** A point located inside a dense region.
- **Border Point:** A point located near the boundary of a dense region.
- **Noise Point:** A point that does not belong to any sufficiently dense region



Parameters of DBSCAN

DBSCAN requires two input parameters:

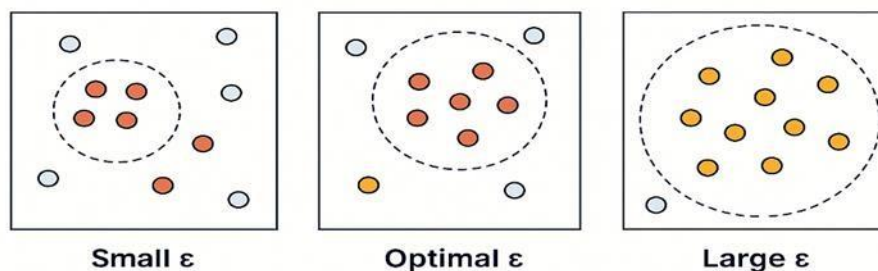
Epsilon (ϵ or Eps)

- Epsilon defines the **maximum neighborhood radius** around a data point. Two observations are considered neighbors when the distance between them is less than or equal to ϵ .
- The value of epsilon directly affects the size and connectivity of the discovered clusters.

Impact of Epsilon

Epsilon Value	Effect
Too small	Neighborhoods become very small, fewer core points are identified, and many observations may be classified as noise
Appropriate	Natural dense regions are identified as meaningful clusters
Too large	Unrelated dense regions may connect and merge into fewer clusters

Effect of Different Epsilon Values



Minimum Points (MinPts)

- MinPts, called `min_samples` in Scikit-learn, defines the minimum number of observations required within an epsilon neighborhood for the selected point to qualify as a **core point**

The selection of MinPts depends on factors such as:

- Dataset characteristics
- Number of dimensions
- Expected level of noise
- Domain knowledge
- Business requirements

In Scikit-learn, the selected point itself is included in the neighborhood count.

Types of Points in DBSCAN

Core Point

A **core point** has at least MinPts observations, including itself, within its epsilon neighborhood.

$$|N_{\epsilon}(p)| \geq MinPts$$

Core points represent dense regions and are responsible for creating and expanding clusters.

Border Point

A **border point** has fewer than MinPts observations within its own epsilon neighborhood but lies within the epsilon neighborhood of a core point.

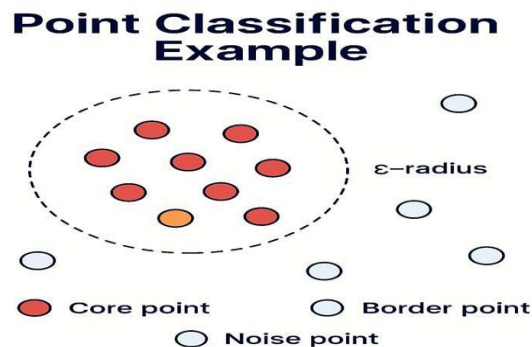
A border point belongs to a cluster but cannot independently create or expand the cluster.

Noise Point

A **noise point** is neither a core point nor a border point. It does not have enough neighboring observations and is not reachable from any core point.

In Scikit-learn, noise points receive the label:

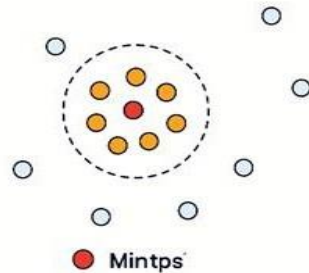
-1



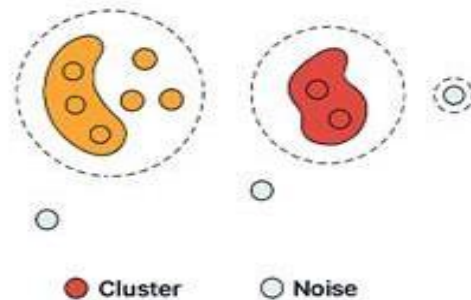
Step-by-Step Working of DBSCAN

- DBSCAN forms clusters using the following process:
 1. Select an unvisited data point.
 2. Find all observations within its epsilon neighborhood.
 3. Check whether the neighborhood contains at least **MinPts** observations.
 4. If the condition is satisfied, classify the selected point as a core point and create a new cluster.
 5. Add neighboring points to the cluster.
 6. Expand the cluster through neighboring core points.
 7. Add reachable border points to the cluster.
 8. If the selected point is not a core point, temporarily classify it as noise.
 9. Repeat the process until all observations have been visited.
 10. Return the final clusters and noise points

DBSCAN on Toy Dataset



Final Result of DBSCAN



Density-Based Concepts

Directly Density-Reachable

A point q is **directly density-reachable** from point p when:

1. Point q lies within the epsilon neighborhood of point p .
2. Point p is a core point.

This relationship allows DBSCAN to begin expanding a cluster from a core point.

Density-Reachable

A point is **density-reachable** from another point when the two points are connected through a sequence of directly density-reachable observations.

A point q is density-reachable from p if there exists a sequence of points $p_1, \dots, p_n$ where $p_1=p$, $p_n=q$, and each point is directly density-reachable from the previous point.

Density-Connected

Two points are **density-connected** when there exists a common point from which both observations are density-reachable.

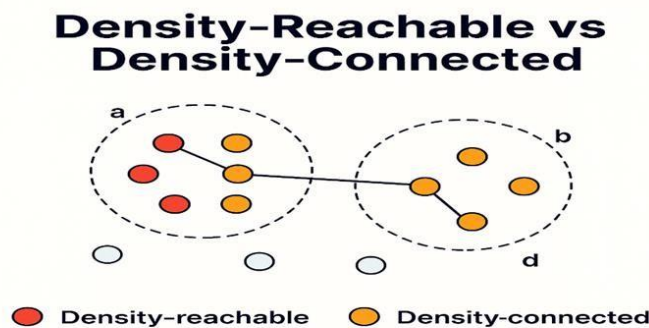
Density connectivity allows DBSCAN to group connected dense regions into the same cluster.

Cluster Definition

In DBSCAN, a cluster is a **maximal collection of density-connected data points**.

This means that:

- All density-connected observations belong to the cluster.
- No additional density-reachable point can be added without already belonging to that cluster.



Optimising Epsilon Using the K-Distance Method

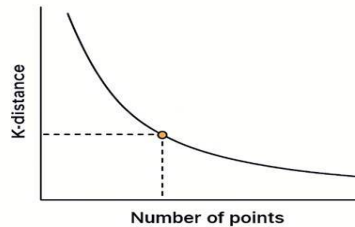
Selecting an appropriate epsilon value is one of the most important steps in DBSCAN.

The **K-distance graph** provides a practical method for estimating a suitable epsilon value.

Steps

1. Select a value of K, commonly related to MinPts.
2. Calculate each observation's distance to its K-th nearest neighbor.
3. Sort the calculated distances in ascending order.
4. Plot the sorted K-neighbor distances.
5. Identify the **knee point**, where the distance begins to increase sharply.
6. Use the distance near the knee as a candidate epsilon value.

K-Distance (Elbow) Graph

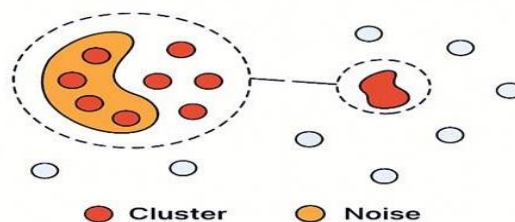


point $\rightarrow$ optimal ε Identify the elbow (knee)

DBSCAN has the following limitations:

- It may struggle when clusters have significantly different densities.
- Selecting an appropriate epsilon value can be difficult.
- The best MinPts value depends on the dataset and problem.
- It is sensitive to feature scaling because it relies on distance calculations.
- Its performance may decrease in high-dimensional datasets because distances become less informative.
- A single global epsilon value may not represent both dense and sparse clusters effectively

DBSCAN with Varying Density



Comparison of Clustering Algorithms

Feature	K-Means	Hierarchical Clustering	DBSCAN
Clustering approach	Centroid-based	Hierarchy-based	Density-based
Typical cluster	Compact or spherical	Depends on linkage and	Arbitrary or irregular



shape		distance metric	
Main parameters	Number of clusters K	Linkage, metric, and cluster cut	eps and MinPts
Number of clusters required	Yes	Often selected using a cut level	No
Outlier handling	No built-in noise detection	Limited	Built-in noise detection
Cluster representation	Centroids	Dendrogram	Dense connected regions
Feature scaling required	Usually yes	Depends on metric	Usually yes
Scalability	Generally high	Often limited for large datasets	Depends on dataset size, dimensions, and neighborhood search

Implementation of DBSCAN in Python

Required Libraries: -

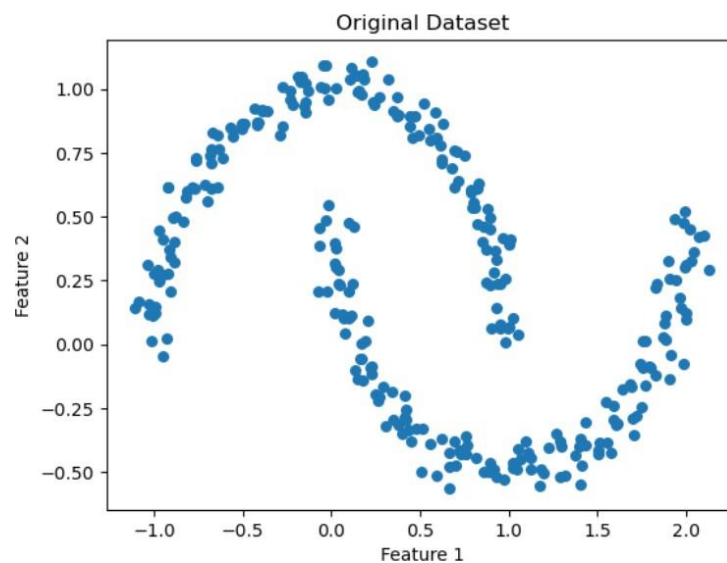
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import DBSCAN
from sklearn.datasets import make_moons
from sklearn.preprocessing import StandardScaler
```

Generate dataset: -

```
X, y = make_moons(n_samples=300, noise=0.05, random_state=42 )
```

Visualisation: -

```
plt.scatter(X[:, 0], X[:, 1],  
s=30) plt.title("Original  
Dataset") plt.xlabel("Feature 1")  
plt.ylabel("Feature 2")  
plt.show()
```



Feature Scaling

```
scaler = StandardScaler()  
X_scaled = scaler.fit_transform(X)
```

Applying DBSCAN

```
dbscan = DBSCAN(eps=0.3, min_samples=5)  
dbscan_labels = dbscan.fit_predict(X_scaled)
```

Understanding Output

```
unique_labels = set(int(label) for label in dbscan_labels)  
print("Cluster labels:", unique_labels)
```

OUTPUT: Cluster labels: {0, 1}



Visulisation :-

```
plt.figure(figsize=(8, 6))

for label in unique_labels:
    if label == -1:
        plt.scatter(
            X_scaled[dbscan_labels == label, 0],
            X_scaled[dbscan_labels == label, 1],
            color='gray',
            s=30,
            label='Noise'
        )
    else:
        plt.scatter(
            X_scaled[dbscan_labels == label, 0], X_scaled[dbscan_labels == label,
1],
            s=30,
            label=f'Cluster {label}'
        )
plt.title("DBSCAN Clustering Result")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.legend()
plt.show()
```



DISCLAIMER: *This learning course may contain images, both still and moving, including photographs, advertisements, third-party trademarks, video clips etc. that may constitute*

copyrighted material procured from open-source, public domain, social media, or other print or digital publications, with the sole and bona fide intention of imparting education, disseminating information and illustrating an academic concept or an issue. We believe that this constitutes a 'fair dealing' exception under Section 52 of the Indian Copyright Act, 1957. Throughout this course, you may see examples of different brands and institutions. These have been chosen from specific cases in a way that would aid in demonstrating concepts. These do not reflect the current performance of any company. Any opinion expressed is strictly for educational purposes.